Object Detection

Object Detection = Image Classification + Object Localization

- Image Classification: The entire image is classified with a single label
  - Input: An image
  - Output: A class label
- Object Localization: Locate presence of objects in an image and indicate their location with a bounding box
  - Input: An image with 1 or more objects
  - Output: 1 or more bounding boxes
- Object Detection: Locate the presence of objects with a bounding box and types or classes the located objects in an image
  - Input: An image with 1 or more objects
  - Output: 1 or more bounding boxes and a class label for each bounding box

Method 1: Traditional object detection approach

- This involves standard, computer-vision based object detection methods (non-deep learning methods) such as **sliding windows** and **image pyramids**
- Typically used in **HOG + Linear SVM-based** object detectors
- Slow, tedious and error-prone
- Steps:
  - Step 1: Fixed size sliding windows, which slide from left-to-right and top-to-bottom to localize objects at different locations
  - Step 2: An image pyramid to detect objects at varaying scales
  - Step 3: Classification of each ROI (Region Of Interest) via:
    - A pre-trained CNN
    - Traditional ML method (like SVM)
  - Step 4: If the classification probability of label L: $P(L) > T$, mark the bounding box of the ROI as the label L

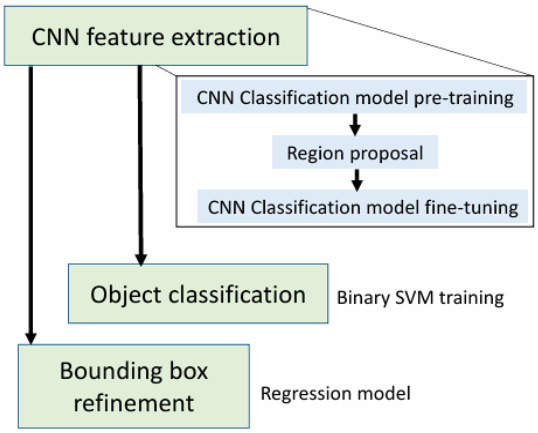
Method 2: Pre-trained network as a base network

- Basically is to treat the pre-trained classification network as a **base network** in DL **object detection framework** (like Faster R-CNN, SSD or YOLO)
- The base networks are the common (classification) CNN architectures, including:
  - VGGNet
  - ResNet
  - MobileNet
  - DenseNet
- Typically these networks are pre-trained to perform classification on a large image dataset, such as ImageNet, to learn a rich set of discerning, discriminating filters

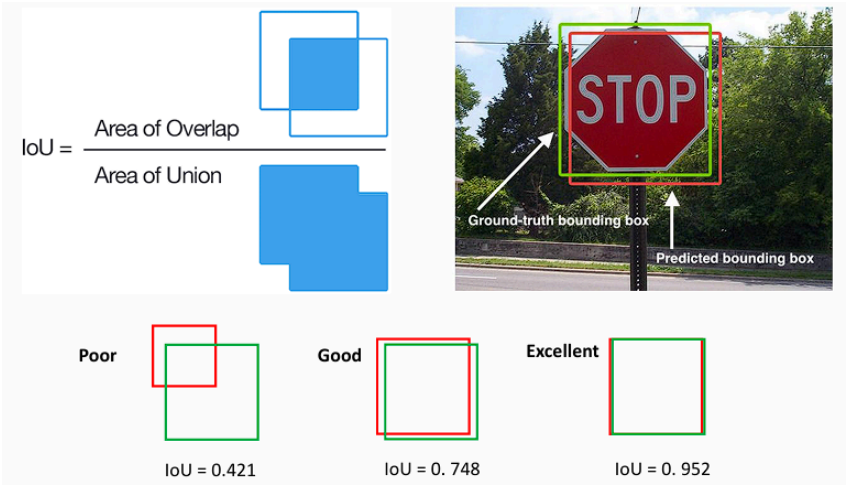
Object Detection framework: Two-stage detector

- First, the model proposes a set of ROI by region proposal methods (selective search, edge boxes) or network
- Then a classifier only processes the region candidates
- Models in the R-CNN are all region-based: RCNN, Fast-RCNN, Faster-RCNN
  → This solution can be slow because we have to run predictions for every selected region

1. RCNN (Region based CNN)



- **CNN Feature extraction:**
  - Pre-train a CNN network on image classification tasks (like VGG or ResNet trained on ImageNet dataset)
  - We first generates region proposals using an algorithm such as Edge Boxes (about 2k candidate proposals per image)
  - Those regions may contain target objects and they are of different sizes
  - Region proposal with $\geq 0.5\ IoU$ (Intersection of Union) overlap with a ground-truth box are treated as positives for that box's class and the rest as negatives



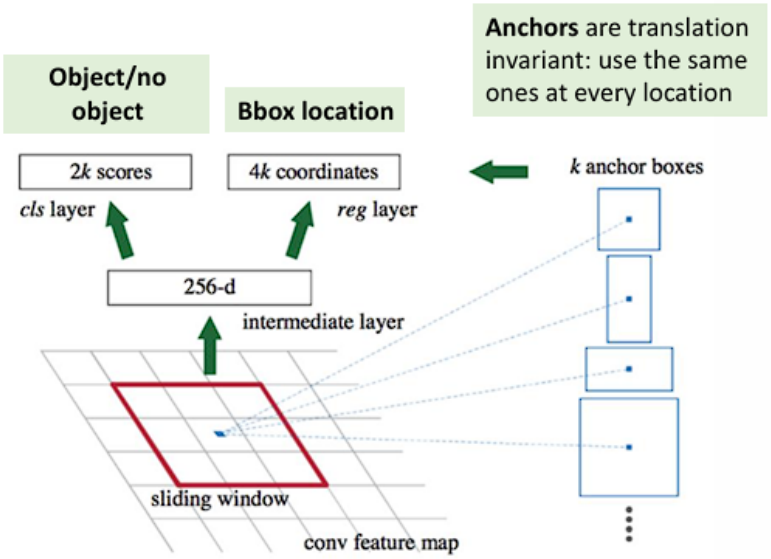
    - The proposed regions are cropped out of the image, resize and warped to have a fixed size as required by CNN
    - Fine-tuning the CNN on the wrapped proposal regions for $K + 1$ classes
    - In the fine-tuning stage, we use a much smaller learning rate and the mini-batch oversamples the positive cases because most proposed regions are just background
- **Object Classification:**
    - Given every image region, 1 forward propagation through the CNN generates a feature vector
    - This vector is then consumed by a binary SVM trained for each class independently
    - The positive samples are proposed regions with IoU overlap threshold ≥ 0.3, and negative samples are irrelevant others
- **Bounding box Regression:**
    - This stage is used to improve the localization performance
    - It predicts a new bounding box for the detection for each object class using the features computed by CNN
    - Only the predicted box with ≥ 0.6 is used to train the class-specific bounding box regressor
    - The bounding box regression is used to adjust an initial, rough guess of an object's location (proposal box) to more accurately match its actual position and size (groundtruth box)
- **Problems with RCNN:**
    - Slow at test-time
        - Due to independent forward passes of the CNN
        - Need to run full forward pass of CNN for each region proposal
    - Post-hoc training of SVMs and regressors
        - CNN features not updated in response to SVMs and regressors
        - This could lead to the generation of bad candidate region proposals
    - Complex multistage training pipeline
        - Training is expensive and slow
        - The whole process involves 3 models separately without much shared computation:
            - CNN for image classification and feature extraction
            - SVM classifier for object classification
            - Regression model for bounding boxes correction

2. Fast-RCNN

- This makes RCNN faster by improving the training procedure by unifying 2 independent models into 1 jointly trained framework and increasing shared computation results
- **Problems:**
    - Problem 1:
        - Slow at test-time due to independent forward passes of the CNN
        - Solution: Share computation of convolutional layers between proposals for an image
    - Problem 2:
        - Post-hoc training: CNN not updated in response to final classifiers and regressors
    - Problem 3:
        - Complex training pipeline
        - Solution: Just train the whole system end-to-end all at once
- **ROI (Region Of Interest) pooling:**
    - It is a type of max-pooling to convert features in the projected region of the image of any size ($h \times w$) into a small fixed window ($H \times W$)
    - The input region is divided into $H \times W$ grids, approximately every subwindow of size $h/H \times w/W$. Then apply max-pooling in each grid
- **Model workflow:**
    - Step 1: Pre-train a CNN on image classification tasks
    - Step 2: Propose regions by selective search
      (about 2k candidates per image)
    - Step 3: Replace the last max-pooling layer of the pre-trained CNN with a ROI pooling layer. The ROI pooling layer outputs fixed-length feature vectors of region proposals
    - Step 4: The model is altered to branch into 2 output layers
        - Softmax estimator of $K + 1$ classes
        - Bounding-box regression model which predicts offsets relative to the original ROI for each of k classes
- **Speed Bottleneck:**
    - Fast-RCNN is much faster in both training and testing time as compared to RCNN
    - However, the improvement is not dramatic because the region proposals are generated separately by another model and that is very expensive

3. Faster-RCNN

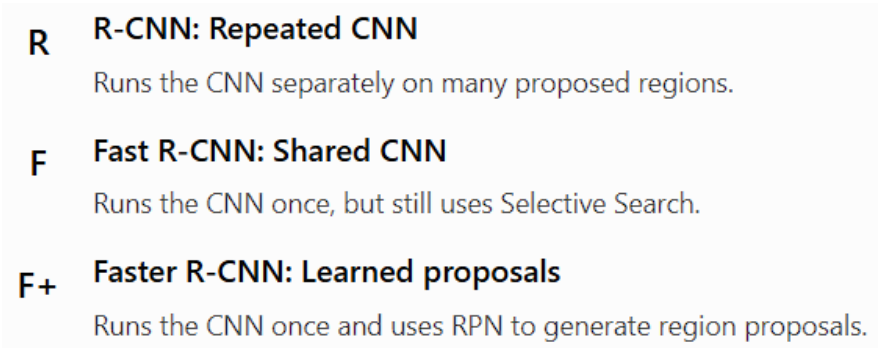
- This inserts a **RPN (Region Proposal Network)** after the last convolutional layer
- The RPN is trained to produce region proposals directly without external proposals
- After RPN, it uses ROI Pooling and an upstream classifier and bounding box regressor just like Fast-RCNN



- The RPN takes all the anchor boxes as input and then generates the object score for each box and performs regression to find a more accurate boundary box
- The Anchor Boxes are predefined and fixed-size boxes, which are spread over the entire image. These boxes come in different sizes and different aspect ratios. This helps to **capture different types of objects**
- RPN works on the feature map (output of CNN), and each feature of this map is called Anchor Point. For each anchor point, we place 9 anchor boxes over the image. These anchor boxes are centered at the point in the image which is corresponding to the anchor point of the feature map
- <u>Training of RPN:</u>
    - Assign a label to each anchor box, based on its IoU (Intersection over Union) with given ground truth. We basically assign either of the three (-1,0,1) labels to each anchor box
    - After assigning the labels, it creates the mini-batch of 256 randomly picked anchor boxes, all of these anchor boxes are picked from the same image
    - The ratio of the number of positive and negative anchor boxes should be 1:1 in the mini-batch, but if there are less than 128 positive anchor boxes then we pad the mini-batch with negative anchor boxes
    - Now the RPN can be trained end-to-end by **backpropagation and SGD** (Stochastic Gradient Descent)

- Training of Faster-RCNN:
    - Trains the end-to-end RPN, where the CNN is initialized by a pre-trained model, which has been trained on ImageNet dataset. This RPN training fine-tune the weights end-to-end for the region proposal task
    - Trains a separate Fast-RCNN, here it also initialized the CNN by pre-trained model (trained on ImageNet). Till now there is no sharing of convolution layers between RPN and Fast-RCNN
    - Now we initialize the RPN from the weights of Fast-RCNN. After initialization, we freeze (exclude from training) the shared convolution layers and fine-tune the layers which are specific to the RPN
    - Finally, we keep the shared convolution layers frozen and fine-tune the layers specific to Fast-RCNN
    - **Joint Training:** 1 network, 4 losses
        - RPN classification (anchor good / bad)
        - RPN regression (anchor → proposal)
        - Fast-RCNN (over classes)
        - Fast-RCNN (proposal → box)

	R-CNN	Fast R-CNN	Faster R-CNN
Region Proposals	Selective Search	Selective Search	RPN
CNN runs	Separately for each region	Once per Image	Once per Image
Feature sharing	No	Yes	Yes
Training	Multi-stage	More unified	More unified



Object Detection framework: One-stage detector

- Single convolutional network predicts the bounding boxes and the class probabilities for these boxes
- Can be YOLO or SDD
  → Commonly used for real-time object detection as they trade a bit of accuracy for large improvements in speed

1. YOLO (You Only Look Once)

- This model is the very first attempt in building a fast real-time object detector
- YOLO is super fast, but it is not good at recognizing irregularly shaped objects or a group of small objects due to a limited number of bounding box candidates
- Mechanism:
  - Divides the image into an $S \times S$ grid
  - For each grid cell, predicts B bounding boxes, confidence for those boxes and C class probabilities
    $S \times S \times B \; bounding \; boxes$
    $Confidence = P(object) \times IoU(prediction, \; truth)$
  - The total prediction values for 1 image is $S \times S \times (B * 5 + C)$, which is the tensor shape of the final conv layer of the model
  - Each bounding box consists of 5 predictions: $x, y, w, h \; and \; confidence$
- Network Architecture:
  - The base model is similar to GoogleNet with inception model replaced by 1×1 and 3×3 conv layers. The final prediction of shape $S \times S \times (5B + C)$ is produced by 2 fully connected layers over the whole conv feature map
  - For example: $S = 7, \; B = 2, \; C = 20 \; for \; 20 \; labelled \; classes$. The final prediction is a $7 \times 7 \times 30$ tensor
- YOLO offers a significant increase in speed over Faster-RCNN which is 45fps vs. 7 fps
- However, YOLO faces a decrease in detection accuracy compared to Faster-RCNN, which is 63.4% vs. 73.2%

2. SSD (Single Shot Detector)

- This is 1 of the first attempt of using **CNN pyramidal feature hierarchy** for efficient detection of objects of various sizes
- The SSD network ran both faster and had superior performance to YOLO
- SSD uses the VGG16 model pre-trained on ImageNet as its base model for extracting useful image features
- On top of VGG16, SSD adds several conv feature layers of decreasing size in which allow predictions of detecting at multiple scales
  (while YOLO use 1 single scale feature map)

- Intuitively large fine-grained feature maps at earlier levels are good at capturing small objects and small coarse-grained feature maps can detect large objects well
- Convolutional predictors for detection:
    - For a feature map of size $m \times n$ with p channels, the basic elements for predicting parameters of a potential detection is a $3 \times 3 \times p$ small kernel that produces either a score for a category or a shape offset relative to the default box coordinates
      (while YOLO has a fully connected layer for a score and a bbox coordinates)
- Matching strategy:
    - SSD predictions are classified as positive matches or negative matches
    - SSD only uses positive matches in calculating the localization cost L_{IoC}
    - If the corresponding **default boundary box** (not the predicted boundary box) has an $IoU > 0.5$ with the ground truth, the match is positive. Otherwise, negative.
    - Once we identify the positive matches, we use the corresponding predicted boundary boxes to calculate the cost
    - This matching strategy nicely partitions what shape of the ground truth that a prediction is responsible for

Object Detection common tricks

1. Hard Negative Mining

- Not all the negative examples are equally hard to be identified. For example, if it holds pure empty background, it is likely an **“easy negative”**
- If the box contains weird noisy texture or partial object, it could be hard to be recognized and these are **“hard negative”**
- We can explicitly find those false positive samples during the training loops and include them in the training data so as to improve the classifier

2. Non-max Suppression (NMS)

- The purpose of non-max suppression is to select the best bounding box for an object and reject or “suppress” all other bounding boxes
- The NMS takes 2 things into account:
    - The confidence score given by the model
    - The IoU of the bounding boxes